

(Sinh viên không được sử dụng tài liệu. Làm bài trực tiếp trên đề)

|                     |      |                |
|---------------------|------|----------------|
| HỌ VÀ TÊN SV: ..... | ĐIỂM | CÁN BỘ COI THI |
| MSSV: .....         |      |                |
| STT: .....          |      |                |
| PHÒNG THI: .....    |      |                |

**BẢNG TRẢ LỜI**

Vui lòng điền tất cả câu trả lời của PHẦN 1 và PHẦN 2 vào bảng dưới đây.

**PHẦN 1. TRẮC NGHIỆM (6 điểm)**

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| Câu 1  | Câu 2  | Câu 3  | Câu 4  | Câu 5  |
| C      | C      | A      | B      | C      |
| Câu 6  | Câu 7  | Câu 8  | Câu 9  | Câu 10 |
| C      | C      | C      | C      | A      |
| Câu 11 | Câu 12 | Câu 13 | Câu 14 | Câu 15 |
| B      | B      | B      | B      | B      |
| Câu 16 | Câu 17 | Câu 18 | Câu 19 | Câu 20 |
| B      | C      | B      | A      | A      |

**PHẦN 2. TỰ LUẬN (4 điểm)**

|           |            |           |           |
|-----------|------------|-----------|-----------|
| Câu 21(a) | Câu 21(b)  | Câu 22(a) | Câu 22(b) |
| 8         | 6          | wait(S);  | BLANK     |
| Câu 22(c) | Câu 22(d)  | Câu 23(a) | Câu 23(b) |
| BLANK     | signal(S); | Vùng 6    | Vùng 1    |

## PHẦN 1. TRẮC NGHIỆM (6 điểm, 0.3 điểm/câu, SV chọn 1 đáp án đúng nhất và điền vào BẢNG TRẢ LỜI)

**Câu 1:** Phát biểu nào sau đây là ĐÚNG khi nói về binary semaphore? (G6.1)

- A. Binary semaphore chỉ nhận các giá trị nguyên lớn hơn hoặc bằng 0
- B. Binary semaphore thường được sử dụng để đảm bảo điều kiện Progress khi đồng bộ
- C. Binary semaphore chỉ nhận giá trị 0 hoặc 1 và thường được dùng cho mục đích đồng bộ hóa
- D. Binary semaphore thường được sử dụng để quản lý số lượng tài nguyên giới hạn

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Binary semaphore (semaphore nhị phân) là biến đồng bộ chỉ nhận một trong hai giá trị là 0 hoặc 1. Nó hoạt động tương tự như một khóa Mutex, thường được dùng để giải quyết bài toán loại trừ tương hỗ (Mutual Exclusion) bảo vệ đoạn găng hoặc đồng bộ hóa thứ tự thực thi giữa các tiến trình.

**Câu 2:** Để cài đặt bộ nhớ ảo, hệ thống cần phải đảm bảo điều kiện nào sau đây? (G2.1)

- A. Hệ điều hành phải quản lý sự di chuyển của trang giữa bộ nhớ chính và bộ nhớ thứ cấp
- B. Phần cứng quản lý bộ nhớ phải hỗ trợ cơ chế phân trang
- C. Tất cả đều đúng

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Để triển khai bộ nhớ ảo thành công, cần sự hỗ trợ kết hợp của cả phần cứng và hệ điều hành: phần cứng (MMU) phải hỗ trợ cơ chế phân trang/phân đoạn và bẫy ngắt lỗi trang (Page Fault trap); hệ điều hành phải có các giải thuật quản lý, cấp phát khung trang và điều khiển việc chuyển đổi dữ liệu (I/O) giữa đĩa và RAM.

**Câu 3:** Cho bảng trang như bên dưới:

|   |   |
|---|---|
| 0 | 1 |
| 1 | 6 |
| 2 | 3 |
| 3 | 4 |

Biết rằng kích thước khung trang là 1KB, hỏi địa chỉ vật lý 4567 là ánh xạ của địa chỉ luận lý nào sau đây? (G2.1)

- A. 3543
- B. 471
- C. 2519
- D. 1496

► **Đáp án đúng: A**

► **Hướng dẫn giải:** Kích thước trang 1 KB = 1024 Bytes. Địa chỉ vật lý PA = 4567:

- Số hiệu khung trang:  $f = \lfloor 4567/1024 \rfloor = 4$ .
- Độ lệch offset:  $d = 4567 \pmod{1024} = 471$ .

Tra bảng trang: Khung trang  $f = 4$  tương ứng với trang luận lý số  $p = 3$ . Do đó địa chỉ luận lý ban đầu là:

$$LA = p \times 1024 + d = 3 \times 1024 + 471 = 3072 + 471 = 3543.$$

**Câu 4:** Critical section trong hệ điều hành là gì? (G 6.1)

- A. Vùng nhớ chứa dữ liệu được chia sẻ bởi hai hay nhiều tiến trình
- B. Vùng code tác động hoặc truy cập dữ liệu được chia sẻ giữa hai hay nhiều tiến trình
- C. Vùng code được dùng chung bởi hai hay nhiều tiến trình
- D. Data section được cấp phát chung cho hai hay nhiều tiến trình

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Đoạn găng (Critical Section) là một đoạn mã nguồn (code) trong đó tiến trình truy cập và/hoặc thao tác làm thay đổi dữ liệu dùng chung (biến toàn cục, tập tin chia sẻ,...). Để tránh xung đột dữ liệu (Race condition), hệ điều hành phải đảm bảo tại một thời điểm chỉ có tối đa một tiến trình được thực thi trong đoạn găng.

**Câu 5:** Cho các phát biểu sau: (G 2.1)

- (1) Hiện tượng phân mảnh ngoại xảy ra khi có tổng dung lượng bộ nhớ trống đủ để thỏa mãn yêu cầu cấp phát, nhưng các vùng nhớ trống này không liên tục.
- (2) Thuật toán First-fit cấp phát vùng nhớ trống có kích thước nhỏ nhất nhưng đủ lớn để chứa tiến trình.
- (3) Kỹ thuật gom cụm (Compaction) là một giải pháp nhằm giảm thiểu hiện tượng phân mảnh ngoại.
- (4) Phân đoạn (Segmentation) là cơ chế quản lý bộ nhớ phản ánh góc nhìn của người lập trình về cấu trúc chương trình.

Số lượng phát biểu ĐÚNG là:

- A. 1                                      B. 2                                      C. 3                                      D. 4

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Xét tính đúng/sai của từng phát biểu:

- (1) ĐÚNG: Đây là định nghĩa chuẩn của hiện tượng phân mảnh ngoại vi (External Fragmentation).
- (2) SAI: Thuật toán cấp phát vùng nhớ trống nhỏ nhất đủ lớn là **Best-fit**. First-fit chỉ cấp phát vùng nhớ trống \*đầu tiên\* tìm thấy đủ lớn.
- (3) ĐÚNG: Gom cụm (Compaction) di chuyển các khối nhớ đã cấp phát về một phía để gom các vùng trống rời rạc thành một khối liên tục duy nhất, giải quyết phân mảnh ngoại.
- (4) ĐÚNG: Phân đoạn chia bộ nhớ theo các đơn vị logic (main program, function, stack, symbol table) phù hợp với tư duy lập trình của con người.

Vậy có đúng 3 phát biểu đúng gồm: (1), (3), (4).

**Câu 6:** Việc cho phép nạp một phần chương trình vào bộ nhớ có ý nghĩa gì? (G 2.1)

- A. Tăng hiệu suất của CPU  
B. Giảm dung lượng của bộ nhớ phụ  
C. Giảm mức độ đa chương của hệ thống  
D. Giảm số lượng tiến trình chờ trong hàng đợi Ready

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Khi chỉ cần nạp một phần chương trình (các trang thực sự cần thiết) vào RAM thay vì toàn bộ tiến trình, bộ nhớ chính sẽ chứa được nhiều tiến trình hơn cùng lúc. Điều này giúp làm tăng **Mức độ đa chương (Degree of Multiprogramming)** của hệ thống và tận dụng CPU hiệu quả hơn.

**Câu 7:** Thời gian truy xuất bộ nhớ hiệu dụng (EAT) của hệ thống phân trang theo yêu cầu được tính như thế nào? (Biết  $p$  là tỉ lệ lỗi trang, thời gian truy xuất bộ nhớ chính là  $M_a$ , thời gian xử lý một lỗi trang là  $PF_t$ ) (G 2.1)

- A.  $EAT = p \times M_a + (1 - p) \times PF_t$   
B.  $EAT = M_a + PF_t$   
C.  $EAT = (1 - p) \times M_a + p \times PF_t$   
D.  $EAT = p \times (M_a + PF_t)$

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Thời gian truy xuất bộ nhớ hiệu dụng là giá trị kỳ vọng (trung bình có trọng số): với xác suất  $(1 - p)$  truy xuất trúng trang trong RAM mất thời gian  $M_a$ ; với xác suất  $p$  phát sinh lỗi trang mất thời gian xử lý  $PF_t$ . Do đó công thức chuẩn là:

$$EAT = (1 - p) \times M_a + p \times PF_t.$$

**Câu 8:** Phát biểu nào sau đây là ĐÚNG khi nói về trạng thái an toàn (safe state) trong giải thuật Banker? (G 6.1)

- A. Trạng thái an toàn đảm bảo hệ thống không bao giờ bị phân mảnh bộ nhớ
- B. Nếu hệ thống ở trạng thái không an toàn thì chắc chắn sẽ xảy ra deadlock ngay lập tức
- C. Trạng thái an toàn là trạng thái mà tại đó hệ thống có thể cấp phát tài nguyên cho tất cả các tiến trình theo một thứ tự nào đó mà không dẫn đến deadlock
- D. Trạng thái an toàn là trạng thái mà tất cả các tài nguyên đều đang rảnh

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Định nghĩa chuẩn của Trạng thái an toàn (Safe state) trong giải thuật Banker: Hệ thống ở trạng thái an toàn nếu tồn tại ít nhất một **Chuỗi an toàn (Safe Sequence)**  $\langle P_1, P_2, \dots, P_n \rangle$  sao cho với mỗi  $P_i$ , các tài nguyên mà nó còn có thể yêu cầu tối đa ( $Need_i$ ) đều có thể được đáp ứng bởi tài nguyên sẵn có cộng với tài nguyên đang giữ bởi các tiến trình hoàn thành trước nó.

**Câu 9:** Giả sử ta muốn giải quyết bài toán producer-consumer bằng việc sử dụng semaphore với một bộ đệm có kích thước giới hạn là  $N$ . Khi đó, các semaphore `mutex`, `empty`, `full` cần được khởi tạo các giá trị ban đầu tương ứng là: (G 6.1)

- A.  $1, 0, N$
- B.  $0, 1, N$
- C.  $1, N, 0$
- D.  $N, 1, 0$

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Trong bài toán Producer - Consumer dùng bộ đệm kích thước  $N$ :

- Semaphore `empty` đếm số ô nhớ trống trong bộ đệm  $\rightarrow$  Khởi tạo bằng  $N$  (ban đầu toàn bộ đệm trống).
- Semaphore `full` đếm số ô nhớ đã có dữ liệu  $\rightarrow$  Khởi tạo bằng 0 (ban đầu chưa có sản phẩm).
- Semaphore `mutex` bảo vệ thao tác ghi/đọc bộ đệm (loại trừ tương hỗ)  $\rightarrow$  Khởi tạo bằng 1.

Vậy bộ giá trị tương ứng (`mutex`, `empty`, `full`) là  $(1, N, 0)$ .

**Câu 10:** Một tiến trình có 124 trang, kích thước mỗi trang là 2MB. Hỏi cần bao nhiêu bit để biểu diễn địa chỉ luận lý của tiến trình này? (G 2.1)

- A. 28 bit
- B. 21 bit
- C. 7 bit
- D. 32 bit

► **Đáp án đúng: A**

► **Hướng dẫn giải:** Cấu trúc địa chỉ luận lý gồm: Số hiệu trang  $p$  và độ lệch offset  $d$ :

- Số hiệu trang: Tiến trình có 124 trang  $\Rightarrow$  Số bit cần dùng là  $k$  sao cho  $2^k \geq 124 \Rightarrow k = \lceil \log_2(124) \rceil = 7$  bit (vì  $2^6 = 64 < 124 \leq 2^7 = 128$ ).
- Độ lệch offset: Kích thước trang là 2 MB  $= 2 \times 2^{20}$  Bytes  $= 2^{21}$  Bytes  $\Rightarrow$  Cần 21 bit cho offset  $d$ .

Tổng số bit của địa chỉ luận lý là:  $7 + 21 = 28$  bit.

**Câu 11:** Một tiến trình đang chạy sẽ chuyển sang trạng thái Waiting khi nào? (G 6.1)

- A. Nó bị hết thời gian sử dụng CPU (hết quantum time)
- B. Nó thực hiện một yêu cầu I/O hoặc chờ một sự kiện nào đó
- C. Nó bị can thiệp bởi một tiến trình có mức độ ưu tiên cao hơn
- D. Nó đã hoàn thành quá trình thực thi

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Tiến trình tự chuyển từ trạng thái Running sang **Waiting** khi nó thực hiện một lời gọi hệ thống yêu cầu I/O (chẳng hạn đọc ổ đĩa, nhận gói tin mạng) hoặc phải chờ đợi một sự kiện/tín hiệu từ tiến trình khác. Khi hết quantum time, tiến trình bị ngắt chuyển về Ready chứ không phải Waiting.

**Câu 12:** Giải thuật thay thế trang nào chọn trang đã ở trong bộ nhớ lâu nhất để thay thế? (G 2.1)

- A. LRU
- B. FIFO
- C. OPT
- D. LFU

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Thuật toán **FIFO** (First-In, First-Out) lựa chọn trang đã được nạp vào bộ nhớ tại thời điểm sớm nhất trong quá khứ (ở trong bộ nhớ lâu nhất) làm trang hy sinh (victim page) để thay thế.

**Câu 13:** Cho biết tỉ lệ tìm thấy trang trong TLB là 85%, thời gian truy xuất TLB là 25ns, thời gian truy xuất bộ nhớ chính là 300ns. Thời gian truy xuất bộ nhớ hiệu dụng (EAT) là: (G 2.1)

- A. 325ns                      B. 370ns                      C. 625ns                      D. 345ns

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Áp dụng công thức EAT có TLB:

$$EAT = t_{tlb} + \alpha \times m + (1 - \alpha) \times 2m$$

Thay số:  $\alpha = 0.85$ ,  $t_{tlb} = 25 \text{ ns}$ ,  $m = 300 \text{ ns}$ :

$$EAT = 25 + 0.85 \times 300 + (1 - 0.85) \times (2 \times 300) = 25 + 255 + 0.15 \times 600 = 25 + 255 + 90 = 370 \text{ ns.}$$

**Câu 14:** Để tránh tình trạng chờ đợi bận (busy waiting) trong Semaphore, hệ điều hành thường chuyển tiến trình sang trạng thái nào và sử dụng các thao tác nào? (G 6.1)

- A. Trạng thái Ready; sử dụng hàm yield() và resume()  
B. Trạng thái Blocked; sử dụng hàm block() và wakeup()  
C. Trạng thái Terminated; sử dụng hàm kill() và start()  
D. Trạng thái Running; sử dụng hàm wait() và signal()

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Để tránh hao tổn chu kỳ xử lý vô ích của CPU khi kiểm tra điều kiện lặp (spin-lock), Semaphore cải tiến đưa tiến trình vào trạng thái **Blocked (Waiting)** bằng hàm nguyên tử block(); khi có tiến trình khác giải phóng tài nguyên, nó được đánh thức trở lại bằng hàm wakeup().

**Câu 15:** Kỹ thuật gom cụm (Compaction) chỉ có thể thực hiện được khi nào? (G 2.1)

- A. Việc nạp trang được thực hiện tại thời điểm biên dịch  
B. Việc liên kết địa chỉ được thực hiện tại thời điểm thực thi (execution time)  
C. Khi hệ thống có đủ bộ nhớ ảo  
D. Khi không có tiến trình nào đang chạy

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Gom cụm (Compaction) đòi hỏi di chuyển các tiến trình đến các vị trí vật lý mới trong RAM khi tiến trình đang tạm dừng chạy. Do đó, việc liên kết địa chỉ phải được thực hiện động tại **Thời điểm thực thi (Execution/Run time)** với sự hỗ trợ của thanh ghi tái định vị (Relocation Register) thì CPU mới tính lại được đúng địa chỉ vật lý sau khi bị di dời.

**Câu 16:** Để giải quyết bài toán đoạn găng, giải pháp cần thỏa mãn bao nhiêu yêu cầu cốt lõi? (G 6.1)

- A. 2 (Loại trừ tương hỗ, Tiến triển)  
B. 3 (Loại trừ tương hỗ, Tiến triển, Chờ đợi có giới hạn)  
C. 4 (Loại trừ tương hỗ, Tiến triển, Chờ đợi có giới hạn, Tiết kiệm tài nguyên)  
D. 1 (Chỉ cần Loại trừ tương hỗ)

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Một giải pháp đúng đắn cho bài toán Đoạn găng bắt buộc phải thỏa mãn đầy đủ **3 yêu cầu**: 1. Loại trừ tương hỗ (Mutual Exclusion); 2. Tiến triển (Progress); 3. Chờ đợi có giới hạn (Bounded Waiting).

**Câu 17:** Chiến lược cấp phát khung trang (frame allocation) nào sau đây chia đều các khung trang cho tất cả các tiến trình bất kể kích thước của chúng? (G 2.1)

- A. Cấp phát theo tỉ lệ (Proportional allocation)    B. Cấp phát động (Dynamic allocation)    C. Cấp phát đều (Equal allocation)    D. Cấp phát ưu tiên (Priority allocation)

► **Đáp án đúng: C**

► **Hướng dẫn giải:** Chiến lược cấp phát đều (**Equal allocation**) chia đều tổng số khung trang rảnh  $m$  cho  $n$  tiến trình, mỗi tiến trình nhận được  $s = m/n$  khung trang như nhau, bất kể tiến trình đó có dung lượng lớn hay nhỏ.

**Câu 18:** Giả sử một Semaphore  $S$  được khởi tạo giá trị là 2. Sau đó, có 6 tiến trình thực hiện thao tác wait( $S$ ) trên semaphore này. Hỏi có bao nhiêu tiến trình bị chặn (chuyển sang trạng thái chờ)? (G 6.1)

- A. 2                                      B. 4                                      C. 6                                      D. 0

► **Đáp án đúng: B**

► **Hướng dẫn giải:** Ban đầu  $S = 2$ :

- 2 thao tác wait( $S$ ) đầu tiên giảm  $S$  xuống lần lượt là 1 và 0  $\Rightarrow$  Cả 2 tiến trình đều thực thi thành công mà không bị chặn.
- 4 thao tác wait( $S$ ) tiếp theo giảm  $S$  xuống lần lượt  $-1, -2, -3, -4 \Rightarrow$  Cả 4 tiến trình này đều bị chặn và đưa vào hàng đợi chờ.

Vậy có đúng **4 tiến trình** bị chặn.

**Câu 19:** Trình tự nào sau đây là ĐÚNG khi chuyển đổi một chương trình từ mã nguồn sang dạng có thể thực thi trong bộ nhớ? (G 2.1)

- A. Mã nguồn  $\rightarrow$  Trình biên dịch (Compiler)  $\rightarrow$  Trình liên kết (Linker)  $\rightarrow$  Trình nạp (Loader)  $\rightarrow$  Bộ nhớ  
 B. Mã nguồn  $\rightarrow$  Trình liên kết (Linker)  $\rightarrow$  Trình biên dịch (Compiler)  $\rightarrow$  Trình nạp (Loader)  $\rightarrow$  Bộ nhớ  
 C. Mã nguồn  $\rightarrow$  Trình nạp (Loader)  $\rightarrow$  Trình biên dịch (Compiler)  $\rightarrow$  Trình liên kết (Linker)  $\rightarrow$  Bộ nhớ  
 D. Mã nguồn  $\rightarrow$  Trình biên dịch (Compiler)  $\rightarrow$  Trình nạp (Loader)  $\rightarrow$  Trình liên kết (Linker)  $\rightarrow$  Bộ nhớ

► **Đáp án đúng: A**

► **Hướng dẫn giải:** Trình tự xử lý chuẩn của hệ thống: Mã nguồn (Source code)  $\xrightarrow{\text{Compiler}}$  Mã đối tượng (Object code)  $\xrightarrow{\text{Linker}}$  Tập tin thực thi (Executable binary)  $\xrightarrow{\text{Loader}}$  Nạp vào bộ nhớ RAM để thực thi.

**Câu 20:** Để đồng bộ hai tiến trình  $P_1$  và  $P_2$  sao cho một câu lệnh  $B$  trong  $P_2$  chỉ được thực thi sau khi câu lệnh  $A$  trong  $P_1$  đã thực thi xong, ta sử dụng một semaphore  $S$ . Giá trị khởi tạo của  $S$  phải là: (G 6.1)

- A. 0                                      B. 1                                      C. 2                                      D. -1

► **Đáp án đúng: A**

► **Hướng dẫn giải:** Để bắt buộc thứ tự  $A$  xong trước  $B$ : Tiến trình  $P_1$  chạy  $A$  rồi gọi signal( $S$ ); Tiến trình  $P_2$  muốn chạy  $B$  phải gọi wait( $S$ ) trước. Để đảm bảo nếu  $P_2$  chạy trước thì bị giữ lại ngay lập tức tại lệnh wait( $S$ ), giá trị khởi tạo của  $S$  bắt buộc phải bằng 0.

## PHẦN 2. TỰ LUẬN (4 điểm)

(Sinh viên ghi rõ các bước tính toán, vẽ bảng minh họa và điền kết quả vào ô tương ứng trong BẢNG TRẢ LỜI)

**Câu 21 (1.5 điểm):** Một hệ thống sử dụng thuật toán thay thế trang FIFO với **4 khung trang** (frames) ban đầu đều trống. Chuỗi truy xuất trang bộ nhớ như sau:

1, 2, 3, 4, 1, 6, 7, 8, 2, 6, 1, 6

- a. Hãy cho biết tổng số lỗi trang (page faults) xảy ra khi thực thi chuỗi truy xuất trên?  
 b. Trang nào bị thay thế (bị đẩy ra khỏi bộ nhớ) nhiều lần nhất?

► **Đáp án đúng: a. 8 lỗi trang; b. Trang 6**

► **Hướng dẫn giải:** Lập bảng chi tiết mô phỏng thuật toán FIFO qua 12 lần truy xuất:

| Bước | Trang | Khung 1 | Khung 2 | Khung 3 | Khung 4 | Lỗi trang? | Ghi chú                     |
|------|-------|---------|---------|---------|---------|------------|-----------------------------|
| 1    | 1     | 1       |         |         |         | Có         | Nạp trang 1 vào Khung 1     |
| 2    | 2     | 1       | 2       |         |         | Có         | Nạp trang 2 vào Khung 2     |
| 3    | 3     | 1       | 2       | 3       |         | Có         | Nạp trang 3 vào Khung 3     |
| 4    | 4     | 1       | 2       | 3       | 4       | Có         | Nạp trang 4 vào Khung 4     |
| 5    | 1     | 1       | 2       | 3       | 4       | Không      | Hit (đã có trang 1)         |
| 6    | 6     | 6       | 2       | 3       | 4       | Có         | Thay trang 1 (vào lâu nhất) |
| 7    | 7     | 6       | 7       | 3       | 4       | Có         | Thay trang 2                |
| 8    | 8     | 6       | 7       | 8       | 4       | Có         | Thay trang 3                |
| 9    | 2     | 6       | 7       | 8       | 2       | Có         | Thay trang 4                |
| 10   | 6     | 6       | 7       | 8       | 2       | Không      | Hit (đã có trang 6)         |
| 11   | 1     | 1       | 7       | 8       | 2       | Có         | Thay trang 6 (vào ở bước 6) |
| 12   | 6     | 1       | 6       | 8       | 2       | Có         | Thay trang 7                |

#### Kết luận:

- a. Tổng số lỗi trang xảy ra là: **8 lần** (tại các bước 1, 2, 3, 4, 6, 7, 8, 9, 11, 12 ... trong đó có 4 lỗi nạp khởi đầu và 4 lỗi thay thế trang; bước 10 là Hit, bước 11 là lỗi trang thứ 7, bước 12 là lỗi trang thứ 8).
- b. Trang 6 là trang được thay vào/đẩy ra tích cực nhất và bị đẩy ra ở bước 11.

**Câu 22 (1.0 điểm):** Hai tiến trình  $P_1$  và  $P_2$  chia sẻ một biến chung  $x$ . Cần đảm bảo rằng tiến trình  $P_2$  phải tính toán xong giá trị của  $x$  trước khi tiến trình  $P_1$  sử dụng  $x$  để tính toán  $y$ . Cho đoạn mã giả với semaphore  $S$  được khởi tạo bằng 0:

Tiến trình  $P_1$ :

```
// (a) .....
y = x / 2;
// (b) .....
```

Tiến trình  $P_2$ :

```
// (c) .....
x = 10 * 2;
// (d) .....
```

Hãy điền các lệnh thích hợp (wait(S);, signal(S);, hoặc BLANK nếu không cần lệnh) vào các vị trí (a), (b), (c), (d) trong bảng trả lời.

► **Đáp án đúng:** (a) wait(S); (b) BLANK; (c) BLANK; (d) signal(S);

► **Hướng dẫn giải:** Yêu cầu bài toán là đồng bộ thứ tự thực hiện: Lệnh  $x = 10 * 2$  của  $P_2$  phải luôn chạy xong trước lệnh  $y = x/2$  của  $P_1$ . Vì  $S$  khởi tạo bằng 0:

- $P_2$  tính xong  $x$ , ngay sau đó phải phát tín hiệu thông báo cho  $P_1$  biết bằng cách gọi signal(S); tại vị trí (d). Trước đó không cần đồng bộ nên vị trí (c) là BLANK.
- $P_1$  muốn đọc giá trị  $x$  thì bắt buộc phải chờ tín hiệu từ  $P_2$ , do đó phải gọi wait(S); tại vị trí (a). Sau khi tính  $y = x/2$  xong không cần làm gì thêm nên vị trí (b) là BLANK.

**Câu 23 (1.5 điểm):** Xét một hệ thống quản lý bộ nhớ liên tục. Danh sách các phân vùng nhớ trống hiện tại theo thứ tự địa chỉ tăng dần gồm:

**Vùng 1:** 340 KB | **Vùng 2:** 120 KB | **Vùng 3:** 250 KB | **Vùng 4:** 410 KB | **Vùng 5:** 190 KB  
| **Vùng 6:** 280 KB

Biết rằng tiến trình vừa được cấp phát gần đây nhất là  $P_3$  và đã được đặt vào **Vùng nhớ 4**. Bây giờ có tiến trình  $P_4$  yêu cầu cấp phát một vùng nhớ có kích thước **240 KB**. Hãy xác định  $P_4$  sẽ được cấp phát vào vùng nhớ nào theo các thuật toán:

- Thuật toán \*\*Next-fit\*\*.
- Thuật toán \*\*Worst-fit\*\*.

► **Đáp án đúng:** a. Vùng nhớ 6; b. Vùng nhớ 1

► **Hướng dẫn giải:** Phân tích từng thuật toán cấp phát phân vùng trống cho  $P_4$  (240 KB):

- a. **Theo giải thuật Next-fit:** Next-fit bắt đầu tìm kiếm từ vị trí phân vùng vừa cấp phát trước đó (Vùng 4). Tìm tiếp tục từ Vùng 5: - Vùng 5 có dung lượng 190 KB < 240 KB → Không đủ dung lượng, bỏ qua. - Duyệt đến Vùng 6 có dung lượng 280 KB ≥ 240 KB → Đủ dung lượng thỏa mãn! ⇒ Cấp phát tiến trình  $P_4$  vào **Vùng nhớ 6**.

- **b. Theo giải thuật Worst-fit:** Worst-fit duyệt qua toàn bộ danh sách các phân vùng trống hiện có và chọn phân vùng có kích thước **lớn nhất** để giảm thiểu việc tạo ra các mảnh vụn nhỏ. Các phân vùng trống hiện tại: Vùng 1 (340 KB), Vùng 2 (120 KB), Vùng 3 (250 KB), Vùng 5 (190 KB), Vùng 6 (280 KB) (vì Vùng 4 đã được cấp cho  $P_3$ ). Phân vùng có kích thước lớn nhất là Vùng 1 với dung lượng 340 KB ( $340 \geq 240$ ).  $\Rightarrow$  Cấp phát tiến trình  $P_4$  vào **Vùng nhỏ 1**.

— HẾT —

Duyệt đề của Bộ môn

Giảng viên ra đề